

README

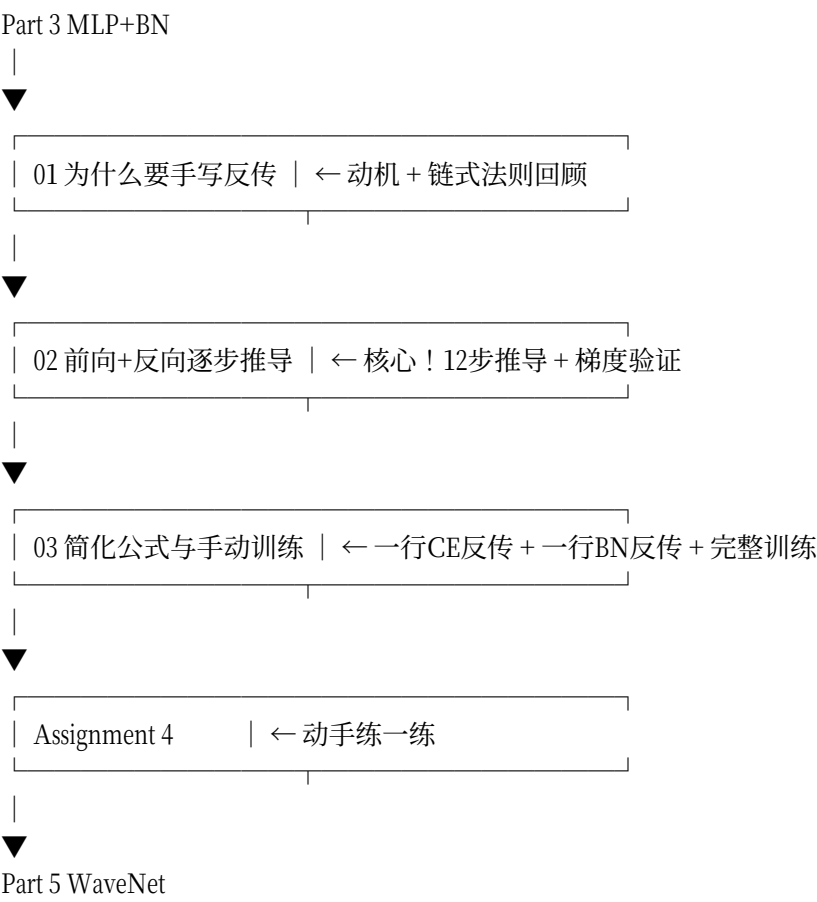
Part 4：手动反向传播

> 把 PyTorch 的 autograd 扒开，看看里面到底在干什么。

章节导航

	配套脚本
	[01_forward_pass_steps.py](../scripts/01_forward_pass_steps.py)
md)	[02_backprop_step_by_step.py](../scripts/02_backprop_step_by_step.py) [03_verify_gradients.py](../scripts/03_verify_gradients.py)
g.md)	[04_cross_entropy_backward.py](../scripts/04_cross_entropy_backward.py) [05_batchnorm_backward.py](../scripts/05_batchnorm_backward.py) [06_manual

学习路线



课后作业

学完本 Part 后，完成 [Assignment 4](../../assignments/assignment_4/)：

1. 实现逐步前向传播函数
2. 实现单步反向传播
3. 实现简化 CrossEntropy 反传
4. 实现简化 BatchNorm 反传
5. （拓展）手动梯度训练完整网络

相关资源

- [Andrej Karpathy - Building makemore Part 4](https://www.youtube.com/watch?v=q8SA3rM6ckI) — 本 Part 的原始视频
- [Part 3 笔记](../../Part3_batchnorm/) — MLP + BatchNorm 架构回顾

学习目标

学完本 Part 你应该能：

- [] 说出 PyTorch autograd 的基本原理（计算图 + 链式法则）
- [] 手动推导 CrossEntropy Loss 的梯度（3 行简化版）
- [] 手动推导 BatchNorm 的梯度（1 行简化版）
- [] 用手动梯度训练完整网络（不用 `loss.backward()`）
- [] 用 `cmp()` 函数验证手写梯度的正确性

[← 上一章：Part 3 BatchNorm](../../Part3_batchnorm/tutorial/README.md) | [下一章：Part 5 WaveNet →](../../Part5_wavenet/tutorial/README.md)

01_why_backprop

01 为什么要手写反向传播？

> "如果你不能手算梯度，你就不算真正理解深度学习。" —— Andrej Karpathy

前置知识：Part 3 MLP+BN 回顾

在 Part 3 里，我们搭了这样一个网络：

输入(context 3个字符)

|



Embedding (每个字符 → 10维向量)

|



拼接 ($3 \times 10 = 30$ 维)

|



训练的时候，我们只需要：

```
python  
loss = F.cross_entropy(logits, Yb)  
loss.backward() # 魔法！自动算出所有梯度
```

一行 `loss.backward()` 就搞定了所有参数的梯度计算。但...这到底是怎么做到的？

为什么要手动推导梯度？

理由 1：理解 autograd 的原理

PyTorch 的 autograd 本质上就是计算图 + 链式法则。当你理解了手写梯度的过程，autograd 就不再是黑魔法，而是理所当然的工具。

理由 2：调试能力

当你的模型不收敛、梯度爆炸/消失、loss 是 NaN... 如果你能手算梯度，就能快速定位问题出在哪一层。

理由 3：实现自定义操作

有时候标准库没有你需要的操作（比如特殊的 loss function、特殊的归一化），你需要自己写前向和反向传播。

理由 4：面试

"请推导 BatchNorm 的反向传播" —— 这是常见面试题。理解了本 Part，你就能自信回答。

链式法则回顾

链式法则是反向传播的数学基础。简单来说：

> 如果 $z = f(y)$ 且 $y = g(x)$, 那么 $dz/dx = dz/dy \times dy/dx$

用计算图来看：

$x \longrightarrow g \longrightarrow y \longrightarrow f \longrightarrow z$

反向传播（从 z 往回推）：

$$dz/dx = dz/dy \times dy/dx$$

$$= (z \text{ 对 } y \text{ 的梯度}) \times (y \text{ 对 } x \text{ 的梯度})$$

多维情况的链式法则

对于矩阵运算 $C = A @ B$ ：

$$dL/dA = dL/dC @ B^T \text{ (梯度传播到 A)}$$

$$dL/dB = A^T @ dL/dC \text{ (梯度传播到 B)}$$

这就是为什么我们在反向传播里到处看到 $@$ 和 $.T$ （转置）。

本 Part 的学习路线

1. 先看前向传播是怎么走的 → 运行 [01_forward_pass_steps.py](../scripts/01_forward_pass_steps.py)
 - 每一步都保存中间变量
 - 打印每步 tensor 的形状
 - 画流程图理解数据流动
2. 然后从 loss 往回推 12 步 → 下节课 02_forward_and_backward.md

小结

概念	一句话总结
前向传播	数据从输入流向输出，每步保存中间变量
反向传播	梯度从 loss 流回参数，每步用链式法则
autograd	自动帮你算链式法则，但本质和你手算一样
为什么要手算	理解原理 + 调试 + 自定义操作 + 面试

下一步 → [02 前向+反向逐步推导](02_forward_and_backward.md)

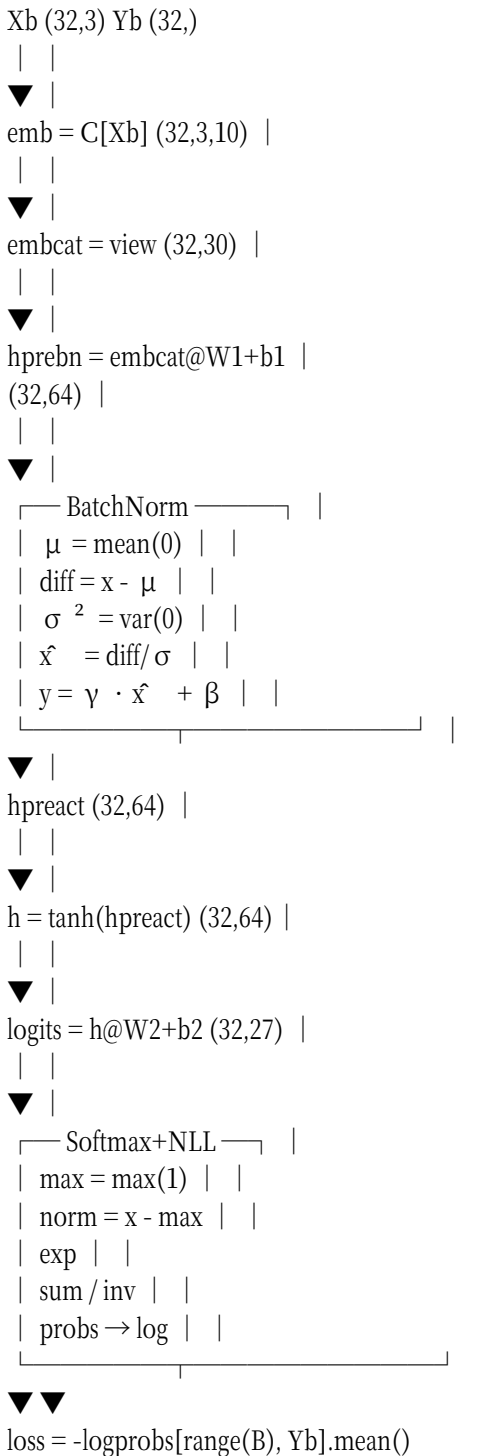
02_forward_and_backward

02 前向 + 反向逐步推导

> 把 MLP+BN 的前向传播展开成计算图，然后一步步从 loss 推回梯度。

前向传播计算图

先看数据在网络中是怎么流动的（以 batch_size=32 为例）：



12 步反向传播推导

核心思想：每步梯度 = 上游梯度 × 局部梯度

> 变量命名约定：代码中的 dX 表示 $\partial L / \partial X$ （loss 对 X 的偏导），不是 dX 的微分。

```

>
> 例如： $\text{dprobs} = \partial L / \partial \text{probs}$ ,  $\text{dlogprobs} = \partial L / \partial \log\text{probs}$ 。
>
> 链式法则的每一步都是： $\partial L / \partial x = \partial L / \partial y \times \partial y / \partial x$ ，对应代码  $\text{dx} = \text{dy} * (\partial y / \partial x)$ 。
> 注意  $\partial y / \partial x$  是局部梯度（只看当前这一步的函数关系）， $\text{dy}$  是上游梯度（从 loss 一路传下来的）。
> 配图参考（Karpathy 原始 notebook 输出）—— dlogits 的热力图，可以看到只有正确类别的位置有非零梯度：
>
> python
> # 可视化：dlogits 热力图 → 生成 ../images/cell018_output01.png
> import matplotlib.pyplot as plt
>
> plt.figure(figsize=(4, 4))
> plt.imshow(dlogits.detach(), cmap='gray')
> plt.colorbar()
> plt.title('dlogits (Gradient of Logits)')
> plt.savefig('../images/cell018_output01.png', dpi=150, bbox_inches='tight')
> plt.show()
>
>
> ![dlogits 热力图](../images/cell018_output01.png)

```

Step 1: loss → dlogprobs

```
loss = -logprobs[range(B), Yb].mean()
```

推导：

- $\text{mean}()$ → 乘 $1/B$
- 负号 → 乘 -1
- 只有 Yb 对应位置有非零梯度

```
python
dlogprobs = torch.zeros_like(logprobs) # (B, 27)
dlogprobs[torch.arange(B), Yb] = -1.0 / B
```

Step 2: dlogprobs → dprobs

```
logprobs = log(probs) →  $d/dx \log(x) = 1/x$ 
```

```
python
dprobs = dlogprobs * (1.0 / probs)
```

Step 3: dprobs → dcounts_sum_inv + dcounts（第一部分）

```
probs = counts × counts_sum_inv（逐元素乘法）
```

乘法求导：两个分支都要传梯度。

```
python
dcounts_sum_inv = (dprobs * counts).sum(1, keepdim=True) # (B,1)
dcounts = dprobs * counts_sum_inv # (B,27)
```

Step 4: dcounts_sum_inv $\rightarrow$ dcounts_sum

$\text{counts_sum_inv} = \text{counts_sum}^{-1} \rightarrow \frac{d}{dx} x^{-1} = -x^{-2}$

```
python
dcounts_sum = dcounts_sum_inv * (-counts_sum -2)
```

Step 5: dcounts_sum $\rightarrow$ dcounts (补充)

```
counts_sum = counts.sum(1, keepdim=True)
```

每个 counts[i,j] 对 counts_sum[i] 的贡献是 1, 所以：

```
python
dcounts += torch.ones_like(counts) * dcounts_sum
```

> ⚠ 注意：dcounts 在 Step 3 已经有值了，这里是累加！

Step 6: dcounts $\rightarrow$ dnorm_logits

$\text{counts} = \text{norm_logits.exp()} \rightarrow \frac{d}{dx} e^x = e^x$

```
python
dnorm_logits = dcounts * counts # counts 就是 e^norm_logits
```

Step 7: dnorm_logits $\rightarrow$ dlogits + dlogit_maxes

```
norm_logits = logits - logit_maxes
```

```
python
dlogits = dnorm_logits.clone() # (B,27)
dlogit_maxes = (-dnorm_logits).sum(1, keepdim=True) # (B,1)
```

Step 8: dlogit_maxes $\rightarrow$ dlogits (补充)

$\text{logit_maxes} = \text{logits.max}(1) \rightarrow$ 只在最大值位置有梯度

```
python
max_indices = logits.argmax(dim=1)
dlogit_maxes_grad = torch.zeros_like(logits)
dlogit_maxes_grad[torch.arange(B), max_indices] = dlogit_maxes.squeeze()
```

```
dlogits += dlogit_maxes_grad
```

> Step 7+8 合起来就是 CE 的简化反传（下节课展开）

Step 9: $dlogits \rightarrow dh + dW2 + db2$

```
logits = h @ W2 + b2 （矩阵乘法 + bias）
```

矩阵乘法的链式法则：

```
python
dh = dlogits @ W2.T # (B, 200) — 传给 h
dW2 = h.T @ dlogits # (200, 27) — 传给 W2
db2 = dlogits.sum(0) # (27,) — 传给 b2
```

Step 10: $dh \rightarrow dhpreact$

$h = \tanh(hpreact) \rightarrow d/dx \tanh(x) = 1 - \tanh^2(x)$

```
python
dhpreact = dh * (1.0 - h ** 2)
```

Step 11: $dhpreact \rightarrow dbngain + dbnbias + dbnraw$

$hpreact = bngain \times bnraw + bnbias$

```
python
dbngain = (dhpreact * bnraw).sum(0, keepdim=True) # (1, 64)
dbnbias = dhpreact.sum(0, keepdim=True) # (1, 64)
dbnraw = dhpreact * bngain # (32, 64)
```

Step 12: $dbnraw \rightarrow dhprebn$ （BatchNorm 反向传播）

这是最复杂的一步，因为 BN 的每一步统计量都依赖于所有样本。

```
bnraw = bndiff * bnvar_inv
bndiff = hprebn - bnmeani
bnvar = bndiff ** 2 .mean(0)
bnvar_inv = (bnvar + ε) ** {-0.5}
bnmeani = hprebn.mean(0)
```

逐步推导（5 个子步骤）：

```
python
12a: dbnraw  $\rightarrow$  dbndiff + dbnvar_inv
```

```
dbndiff = dbnraw * bnvar_inv
```



```
dbnvar_inv = (dbnraw * bndiff).sum(0, keepdim=True)
```

12b: dbnvar_inv $\rightarrow$ dbnvar

```
dbnvar = dbnvar_inv (-0.5) (bnvar + 1e-5) -1.5
```

12c: dbnvar $\rightarrow$ dbndiff2

```
dbndiff2 = torch.ones_like(bndiff2) * (dbnvar / batch_size)
```

12d: dbndiff2 $\rightarrow$ dbndiff (补充)

```
dbndiff += dbndiff2 2.0 bndiff
```

12e: dbndiff $\rightarrow$ dbnmeani + dhprebn

```
dbnmeani = -dbndiff.sum(0, keepdim=True)
```

```
dhprebn = dbndiff + torch.ones_like(hprebn) * (dbnmeani / batch_size)
```

Extra: dhprebn $\rightarrow$ dW1, db1, demb, dC

```
python
dembscat = dhprebn @ W1.T
dW1 = dembscat.T @ dhprebn
db1 = dhprebn.sum(0)
demb = dembscat.view(emb.shape)
dC = torch.zeros_like(C)
for i in range(B):
    for j in range(block_size):
        dC[Xb[i,j]] += demb[i,j]
```

梯度验证

手写梯度到底对不对？用 autograd 当标准答案来比：

```
python
def cmp(name, dt, t):
    """比较手动梯度 dt 和 autograd 梯度 t"""
    exact = torch.allclose(dt, t, atol=1e-5)
    maxdiff = (dt - t).abs().max().item()
    print(f'{name} {" " if exact else " " } {name:15s} | max diff = {maxdiff:.2e}')

```

> 运行 [03_verify_gradients.py](../scripts/03_verify_gradients.py) 查看完整验证结果。

>

> 所有参数的梯度误差应该 < 1e-5

配套脚本

脚本	内容
[01_forward_pass_steps.py](../scripts/01_forward_pass_steps.py)	逐步展开前向传播
[02_backprop_step_by_step.py](../scripts/02_backprop_step_by_step.py)	12 步反向传播
[03_verify_gradients.py](../scripts/03_verify_gradients.py)	梯度验证工具

课后练习

1. 修改网络：把 n_hidden 从 64 改成 100，重新跑前向+反向。观察哪些梯度形状变了？哪些没变？
2. 梯度消失：把 W2 的初始化改大 10 倍（1.0 而不是 0.1），观察 dhpreact 的分布。会发生什么？为什么？
3. 不加 logit_maxes：前向传播时不减最大值（直接 counts = logits.exp()），会发生什么？为什么 Karpathy 要减最大值？

下一步 → [03 简化公式与手动训练](03_simplified_and_training.md)

03_simplified_and_training

03 简化公式与手动训练

> 12 步太多？一行代码就能搞定 CrossEntropy 和 BatchNorm 的反传！

CrossEntropy 简化反传

前面我们用 8 步（Step 1-8）才从 loss 算到 dlogits。但其实整个 CrossEntropy 的反向传播可以简化成 3 行代码

```
python
dlogits = F.softmax(logits, 1) # 先算 softmax
dlogits[range(n), Yb] -= 1 # 正确类别位置减 1
dlogits /= n # 除以 batch size
```

为什么？

CrossEntropy Loss 对 logits 的梯度有一个优雅的解析解：

$$\partial L / \partial \text{logits}_i = \text{softmax}(\text{logits})_i - \quad (i == \text{correct_class})$$

再除以 N（因为 loss 取了 mean）。

直觉理解

- softmax 输出的是概率分布 → 模型给每个类别的"信任度"
- 正确类别位置减 1 → "你对正确答案的信心还不够，要再加把劲"

- 其他位置就是概率值本身 → "你对错误答案太有信心了，要压下去"
- 整体效果：正确类别概率 ↑，其他类别概率 ↓

> 运行 [04_cross_entropy_backward.py](../scripts/04_cross_entropy_backward.py) 查看验证 + 热力图可视化

BatchNorm 简化反传

BatchNorm 的逐步反传（Step 12）有 5 个子步骤，但也可以压缩成 一行公式

```
python
dhprebn = bngain bnvar_inv / n (
n * dhpreact
• dhpreact.sum(0)
• n / (n - 1) bnraw (dhpreact * bnraw).sum(0)
)
```

公式拆解

三个项分别代表：

项	含义
$n * dhpreact$	直接传播（因为 $\hat{x}$ 包含了 x ）
$- \sum (dhpreact)$	减均值 μ 的修正（ μ 依赖于所有样本）
$- n/(n-1) \cdot \hat{x} \cdot \sum (dhpreact \cdot \hat{x})$	除标准差 σ 的修正

其中 $n/(n-1)$ 是 Bessel 校正（用 batch 方差的无偏估计）。

> 运行 [05_batchnorm_backward.py](../scripts/05_batchnorm_backward.py) 查看验证

完整手动训练

把前面的所有简化公式串起来，就能用 手动梯度 训练整个网络！

手动反向传播清单

```
1  CrossEntropy 反传（3 行）
dlogits = softmax(logits)
dlogits[正确位置] -= 1
dlogits /= n

2  Linear 2 反传
dh = dlogits @ W2.T
dW2 = h.T @ dlogits
db2 = dlogits.sum(0)

3  Tanh 反传
dhpreact = dh * (1 - h ^ 2 )

4  BatchNorm 反传（1 行）
```

dhprebn = ... (公式见上)

5 Linear 1 反传

dembcat = dhprebn @ W1.T

dW1 = embcat.T @ dhprebn

6 Embedding 反传

dC[Xb] += demb (scatter 操作)

训练循环

python

for step in range(max_steps):

前向传播（展开所有中间变量）

emb = C[Xb]

hprebn = embcat @ W1

... BatchNorm ...

h = tanh(hpreact)

logits = h @ W2 + b2

loss = cross_entropy(logits, Yb)

手动反向传播（不用 loss.backward() ！）

dlogits = softmax(logits); dlogits[range(n), Yb] -= 1; dlogits /= n

... 其余梯度 ...

参数更新

C.data -= lr * dC

W1.data -= lr * dW1

...

> 运行 [06_manual_training.py](../scripts/06_manual_training.py) 查看完整训练过程 + 采样生成

课后作业

学完本 Part 后，完成 [Assignment 4](../assignments/assignment_4/)：

题目	内容	难度	
1	forward_pass	实现逐步前向传播函数	
2	backward_step	实现单步反向传播	
3	cross_entropy_backward	实现简化 CE 反传	
4	batchnorm_backward	实现简化 BN 反传	

题目	内容	难度	
5	manual_train (拓展)	手动梯度训练	

本 Part 总结

你学到了什么	关键公式
前向传播展开	每步保存中间变量
12 步反传	链式法则 $\times 12$
CE 简化反传	softmax $\rightarrow$ 减1 $\rightarrow$ 除N
BN 简化反传	一行公式 (含 Bessel 校正)
手动训练	不用 <code>loss.backward()</code> 也能训练

下一课预告：Part 5 WaveNet

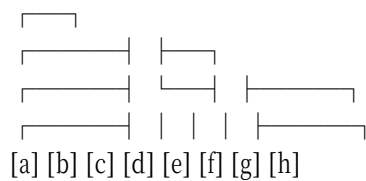
Part 4 的网络用固定的 3 个上下文字符预测下一个字符。但如果上下文更长呢？

Part 5 将引入 WaveNet 架构：

- 用层次化的方式处理越来越大的上下文
- 从 "3个字符 $\rightarrow$ 1个预测" 升级到 "8+个字符 $\rightarrow$ 1个预测"
- 引入 dilated causal convolution 的思想

Part 4: [a][b][c] $\rightarrow$ Linear $\rightarrow$... $\rightarrow$ 下一个字符

Part 5: [a][b][c][d][e][f][g][h] $\rightarrow$ WaveNet $\rightarrow$... $\rightarrow$ 下一个字符



做完作业了吗？ $\rightarrow$ [\[Assignment 4\]\(../../assignments/assignment_4/\)](#)